**Ideation Phase**

**Brainstorm G Idea Prioritization Template**

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 4 Marks |

Here is your Brainstorm C Idea Prioritization Template updated to perfectly match your project title: **Import Data using Transform Maps (Spreadsheet)**.

I have replaced the Incident Management/Client Script concepts with accurate ServiceNow data import concepts (Import Sets, Transform Maps, Coalesce, etc.).

**Brainstorm G Idea Prioritization Template:**

**Step 1: Team Gathering, Collaboration and Select the Problem Statement**

**Selected Problem:**

Bulk inserting or updating records manually from spreadsheets into ServiceNow is time-consuming, highly susceptible to human error, and often results in duplicate or poorly formatted data.

**Key Issues Identified:**

- Manual data entry for hundreds/thousands of records is inefficient and drains resources.
- Risk of creating duplicate records if existing data is not checked before entry.
- Spreadsheet data formats (e.g., dates, text casing) may not natively match ServiceNow field types.
- Spreadsheets may contain invalid data for Reference fields or Choice lists.
- Missing required fields in the spreadsheet can cause incomplete records in the target table.

**Step 2: Brainstorm, Idea Listing and Grouping**

| Category | Brainstormed Idea |
|---|---|
| **Data Ingestion** | Load the raw Excel/CSV spreadsheet into a temporary staging table before touching the live database. |
| **Field Mapping** | Auto-map matching column names from the spreadsheet to the target table fields, and manually map the rest. |
| **Duplicate Prevention** | Identify a unique column (like User ID or Email) to update existing records instead of creating new ones. |
| **Data Transformation** | Use scripts to clean or format incoming data (e.g., standardizing date formats or capitalizing names) during the import. |
| **Reference Handling** | Decide what happens if a spreadsheet value doesn't exist in a dropdown (e.g., create a new choice, ignore it, or reject the row). |
| **Error Handling** | Log errors for any rows missing mandatory target data so they can be fixed later. |
| **Testing** | Load a small test file, run the transform, and verify if records were successfully inserted or updated without data corruption. |

**Step 3: Idea Prioritization**

The ideas can then be grouped according to their implementation approach:

| Priority Area | Selected Idea | ServiceNow Feature |
|---|---|---|
| **Data Staging** | Load spreadsheet data into a temporary staging area. | **Data Source G Import Set Table** |

| Priority Area | Selected Idea | ServiceNow Feature |
| --- | --- | --- |
| **Data Mapping** | Link source spreadsheet columns to the correct target system fields. | **Transform Map G Mapping Assist** |
| **Duplicate Prevention** | Update existing records based on a unique identifier instead of duplicating. | **Coalesce Field** |
| **Reference Data** | Define behavior when spreadsheet data doesn't match existing reference fields. | **Choice Action** (Create/Ignore/Reject) |
| **Data Formatting** | Programmatically format or validate incoming data before it hits the target table. | **Transform Scripts** (onBefore / Field Level) |
| **Execution G Monitoring** | Run the import and monitor for row errors or skipped records. | **Transform History / Import Logs** |

**Project Planning Phase**

**Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)**

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 5 Marks |

**Product Backlog, Sprint Schedule, and Estimation**

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| **Sprint-1** | Data Ingestion C Basic Setup | USN-1 | As a System Admin, I can set up a Data Source and load a spreadsheet into an Import Set (staging table). | 1 | High | Developer A |
| **Sprint-1** | Data Ingestion C Basic Setup | USN-2 | As a System Admin, I can create a Transform Map and auto-map matching spreadsheet columns to | 2 | High | Developer A |

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| | | | target table fields. | | | |
| **Sprint-1** | Data Integrity C Duplicate Prevention | USN-3 | As a Data Manager, I can configure a Coalesce Field so the system updates existing records instead of duplicating them. | 3 | High | Developer B |
| **Sprint-1** | Data Integrity C Duplicate Prevention | USN-4 | As a Data Manager, I can define Choice Action behaviors (create/ignore/reject) for reference fields with un-matched data. | 2 | Medium | Developer B |
| **Sprint-2** | Advanced Data Transformation | USN-5 | As a Developer, I want to write an onBefore Transform Script to standardize incoming date | 3 | High | Developer A |

| Sprint | Functional Requirement (Epic) | User Story Number | User Story / Task | Story Points | Priority | Team Members |
|---|---|---|---|---|---|---|
| | | | formats before insertion. | | | |
| **Sprint-2** | Advanced Data Transformation | USN-6 | As a Developer, I want to write a Field-level mapping script to appropriately capitalize First and Last names. | 2 | Medium | Developer B |
| **Sprint-2** | Testing C Validation | USN-7 | As a QA Lead, I want to run a test import, review Transform logs, and verify target data integrity without errors. | 3 | High | QA Team |

**Project Tracker, Velocity G Burndown Chart**

| Sprint | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed | Sprint Release Date (Actual) |
|---|---|---|---|---|---|---|
| **Sprint-1** | 8 | 6 Days | 01 Feb 2025 | 06 Feb 2025 | 8 | 06 Feb 2025 |

| Sprint | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed | Sprint Release Date (Actual) |
|---|---|---|---|---|---|---|
| **Sprint -2** | 8 | 6 Days | 08 Feb 2025 | 13 Feb 2025 | 8 | 13 Feb 2025 |

**Velocity G Iteration Metrics:**

Total Story Points Completed = 16 (8 Points in Sprint 1 + 8 Points in Sprint 2)

Number of Sprints = 2

Velocity = Total Story Points Completed / Number of Sprints = 16 / 2 = 8 Story Points per Sprint

Average Velocity (AV) per Iteration Unit (story points per day for 6-day sprint duration):

AV = Velocity / Sprint Duration = 8 / 6 = 1.33 Story Points per Day

**Burndown Chart G Execution Notes:**

A burndown chart represents the remaining work (story points) over the sprint duration. For the Import Sets C Transform Maps project, Sprint 1 started with 8 points and burnt down to 0 points by Day 6 as the Data Source setup, Transform Map mappings, and Coalesce configurations were delivered. Sprint 2 started with 8 points covering Transform Scripts for data formatting, Choice Action rules, and E2E import log validation, successfully reaching 0 points on Day 6.

# Project Design Phase-II
# Solution Requirements (Functional G Non-functional)

| | |
|---|---|
| **Date** | 30 September 2026 |
| **Team ID** | SWTID-2026-7082 |
| **Project Name** | Import Data using Transform Maps (Spreadsheet) |
| **Maximum Marks** | 4 Marks |

**Here are the Functional Requirements, Non-Functional Requirements, and Solution Summary adapted for your project: Import Data using Transform Maps (Spreadsheet).**

**Functional Requirements**

**Following are the functional requirements derived from the stated ServiceNow Data Source and Transform Map scope.**

| FR No. | Functional Requirement (Epic) | Sub Requirement (Story / Sub-Task) |
|---|---|---|
| FR-1 | Data Ingestion | Authorized users can upload a spreadsheet file (Excel/CSV) into a configured ServiceNow Data Source. |
| FR-2 | Data Staging | System dynamically loads the raw spreadsheet data into a temporary Import Set (staging) table. |
| FR-3 | Data Mapping | System links staging table columns to target table fields based on configured Transform Map rules and Mapping Assist configurations. |

| FR No. | Functional Requirement (Epic) | Sub Requirement (Story / Sub-Task) |
|---|---|---|
| FR-4 | Data Transformation G Logic | System executes Coalesce rules (to prevent duplicates), Choice Action rules, and Transform Scripts (to format or validate data) during processing. |
| FR-5 | Record Handling | System securely inserts new records or updates existing records in the final target table based on the mapped data. |
| FR-6 | Administrator Maintenance | Authorized administrators can maintain Transform Maps, adjust mapping rules, and review Transform History logs to troubleshoot row errors. |

**Non-functional Requirements**

| FR No. | Non-Functional Requirement | Description |
|---|---|---|
| NFR-1 | Usability | The data mapping interface (Mapping Assist) and Import Logs should provide clear visual cues for mapping fields and identifying specific row errors. |
| NFR-2 | Security | Only authorized ServiceNow administrators with appropriate roles (e.g., import_admin) should be able to upload data, configure Transform Maps, and execute imports into target tables. |
| NFR-3 | Reliability G Data Integrity | The transformation process must consistently prevent duplicate record creation and safely handle mismatched reference data without corrupting the live database. |

| FR No. | Non-Functional Requirement | Description |
|---|---|---|
| NFR-4 | Performance | Bulk data transformations should be processed efficiently in batches through the Import Set engine to minimize any performance degradation on the live ServiceNow instance. |
| NFR-5 | Availability | The data import solution must depend on the availability of the ServiceNow platform's native import/export services. |
| NFR-6 | Scalability | The solution should utilize ServiceNow platform capabilities to handle varying file sizes (from small lists to hundreds of thousands of rows) without requiring a separate, locally managed middleware server. |

**Solution Summary**

**The proposed solution combines ServiceNow Data Sources with Import Sets and Transform Maps. The Import Set handles the secure staging of raw spreadsheet data, while the Transform Map and associated Transform Scripts handle accurate field mapping, duplicate prevention (Coalesce), and dynamic data formatting. Together, these components provide a controlled, reliable data ingestion pipeline and support consistent, high-quality bulk record processing directly into the ServiceNow database.**

# Acceptance Testing

## *UAT Execution & Report Submission*

| | |
|---|---|
| **Date** | 30 September 2026 |
| **Team ID** | SWTID-2026-7082 |
| **Project Name** | Import Data using Transform Maps (Spreadsheet) |
| **Maximum Marks** | 4 Marks |

### 1. Purpose of Document

The purpose of this document is to briefly explain the test coverage and open issues of the Import Data using Transform Maps (Spreadsheet) project at the time of the release to User Acceptance Testing (UAT). The project validates server-side automated data ingestion and mapping rules in ServiceNow, including Data Source configuration, Import Set staging, Transform Map field mapping, duplicate prevention (Coalesce), and scripted data formatting.

### 2. Defect Analysis

This report shows the number of resolved or closed bugs at each severity level, and how they were resolved during the ServiceNow testing phase:

| Resolution | Severity 1 (Critical) | Severity 2 (High) | Severity 3 (Medium) | Severity 4 (Low) | Subtotal |
|---|---|---|---|---|---|
| **By Design** | 0 | 0 | 1 | 1 | 2 |

| Resolution | Severity 1 (Critical) | Severity 2 (High) | Severity 3 (Medium) | Severity 4 (Low) | Subtotal |
|---|---|---|---|---|---|
| **Duplicate** | 0 | 0 | 1 | 0 | 1 |
| **External** | 0 | 0 | 0 | 0 | 0 |
| **Fixed** | 2 | 3 | 4 | 2 | 11 |
| **Not Reproduced** | 0 | 0 | 1 | 0 | 1 |
| **Skipped** | 0 | 0 | 0 | 0 | 0 |
| **Won't Fix** | 0 | 0 | 0 | 1 | 1 |
| **Totals** | 2 | 3 | 7 | 4 | 16 |

**3. Test Case Analysis**

This report shows the number of test cases that have passed, failed, and remained untested across all functional areas of the project:

| Section | Total Cases | Not Tested | Fail | Pass |
|---|---|---|---|---|
| **Data Source Setup G File Ingestion** | 4 | 0 | 0 | 4 |
| **Transform Map Field Mapping (Mapping Assist)** | 3 | 0 | 0 | 3 |
| **Coalesce Validation (Duplicate Prevention)** | 3 | 0 | 0 | 3 |

| Section | Total Cases | Not Tested | Fail | Pass |
|---|---|---|---|---|
| **Choice Action Rule Enforcement (Reference Data)** | 2 | 0 | 0 | 2 |
| **Transform Script Execution (Data Formatting)** | 3 | 0 | 0 | 3 |
| **End-to-End Import Logic G Log Verification** | 3 | 0 | 0 | 3 |
| **Totals** | 18 | 0 | 0 | 18 |

# Project Design Phase-II

## Technology Stack (Architecture G Stack)

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 4 Marks |

**Here is the Technical Architecture mapped to your project: Import Data using Transform Maps (Spreadsheet).**

**Technical Architecture**

**The Deliverable includes the architectural diagram below and the information as per Table 1 G Table 2.**

**Project Title: Import Data using Transform Maps (Spreadsheet)**

**Table-1: Components G Technologies:**

| S.No | Component | Description | Technology |
|---|---|---|---|
| 1. | User Interface | How users (Administrators/Data Managers) interact with the application to upload files and configure mappings. | ServiceNow Core UI (Forms, Lists, Mapping Assist UI) |

| S.No | Component | Description | Technology |
|---|---|---|---|
| 2. | Application Logic-1 | Logic for parsing and loading raw external files into the system staging area. | ServiceNow Data Sources (GlideImportSetLoader) |
| 3. | Application Logic-2 | Logic for matching source columns to destination fields and defining behaviors for reference data. | ServiceNow Transform Maps G Choice Action Configurations |
| 4. | Application Logic-3 | Logic for identifying and updating existing records to prevent data duplication. | ServiceNow Coalesce Field Engine |
| 5. | Database (Staging) | Temporary storage for raw ingested data before processing. | ServiceNow Relational Database (Import Set Tables - e.g., u_imp_spreadsheet) |
| 6. | Database (Target) | Final storage for verified and transformed data. | ServiceNow Relational Database (Target Tables - e.g., incident, sys_user) |
| 7. | File Storage | Storage mechanism for the uploaded spreadsheet files before they are ingested. | ServiceNow Attachment Table (sys_attachment) |
| 8. | Server-side Scripting | Custom programmatic logic executed on the server to validate, | JavaScript (ServiceNow Rhino Engine - Transform Scripts) |

| S.No | Component | Description | Technology |
|---|---|---|---|
| | | format, or reject data on the fly. | |
| G. | Infrastructure (Server / Cloud) | Application Deployment and processing environment. | ServiceNow Cloud (PaaS) - Automated Database instances |

Table-2: Application Characteristics:

| S.No | Characteristics | Description | Technology |
|---|---|---|---|
| 1. | Security Implementations | Access controls ensuring only authorized users can upload data and modify database tables. | ServiceNow Role-Based Access Control (RBAC), ACLs (Access Control Lists) |
| 2. | Scalable Architecture | The solution's ability to handle data sets of varying sizes. | ServiceNow Batch Processing (Import Sets queue and process rows in batches, preventing UI lockups) |
| 3. | Data Integrity | Mechanisms to ensure the live database is not corrupted by malformed external data. | Import Set Staging Buffer, Coalesce Duplicate Prevention, Scripted Validations |

| S.No | Characteristics | Description | Technology |
|---|---|---|---|
| 4. | Availability | Justify the availability of the application. | Inherits ServiceNow's High Availability (HA) cloud architecture (Active-Active server clusters) |
| 5. | Performance | Design consideration for the performance of the application during large bulk imports. | Server-side Asynchronous Processing; imports run in the background without impacting concurrent user performance. |

# Project Design Phase

# Solution Architecture

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 4 Marks |

### 1. Solution Architecture Overview

Solution architecture bridges the gap between business problems (inefficient, risky manual data entry and formatting inconsistencies) and technology solutions (ServiceNow Import Sets and Transform Maps). Its core goals in this project are to:

- Find the optimal data ingestion and mapping controls to enforce data integrity and prevent duplicates during bulk spreadsheet uploads.
- Define the structure, behavior, and interaction rules of ServiceNow Data Sources, Import Set tables, and Transform logic for project stakeholders.
- Provide complete technical specifications according to which data mapping, coalesce rules, and transform scripts are configured, tested, and deployed.

### 2. System Component Architecture

The solution architecture operates across four distinct operational layers within the ServiceNow platform:

#### A. Data Ingestion Layer (External to ServiceNow)

- Provides the entry point through which raw external data enters the ServiceNow ecosystem:

- **Spreadsheet File:** The source file (Excel .xlsx or CSV) containing raw, unformatted data headers and rows.

- **Data Source:** The ServiceNow configuration that defines the file type, sheet number, and header row, instructing the system on how to parse the incoming file.

**B. Staging Layer (ServiceNow Import Set)**

- Acts as a secure, temporary buffer between the raw file and the live database:

  - **Import Set Table:** An auto-generated, temporary table (e.g., u_imp_spreadsheet_data) that stores the exact rows and columns from the spreadsheet before any transformation logic is applied.

**C. Transformation Logic Layer (ServiceNow Transform Engine)**

- Executes server-side mapping engines and script evaluation logic during the data load:

  - **Transform Map Engine:** Evaluates declarative Field Maps linking the Import Set staging table columns to the final Target table fields (via Mapping Assist).

  - **Coalesce Engine (Duplicate Prevention):** Evaluates configured Coalesce fields (e.g., User ID, Asset Tag) against the target table. If a match is found, it updates the existing record; if no match is found, it inserts a new record.

  - **Transform Scripts Engine:** Executes custom server-side scripts (e.g., onBefore, onAfter, Field-level) to programmatically standardize text

casing, format dates, or conditionally skip rows containing invalid data (using ignore = true).

**D. Data G Integration Layer (ServiceNow Database)**

- The ServiceNow relational database backing the final Target table (e.g., User sys_user, Incident incident, or Asset alm_asset). Holds verified, high-quality records that have passed all Transform Map mapping and scripting rules prior to the final database commit.

**3. Solution Architecture G Data Flow Diagram**

The following architectural matrix details the data flow, trigger events, execution sequence, and decision outcomes across the import processing components:

| Trigger Event | Component Engaged | Logic Executed | Action / Outcome |
|---|---|---|---|
| **Administrator uploads Excel/CSV spreadsheet** | **Data Source G Load Data** | Parses the file formatting (headers, delimiters) based on the Data Source configuration. | • Ingests raw data. |

• Populates temporary Import Set staging table. |

| **Transform execution is triggered** | **Transform Map (Field Mapping)** | Links mapped columns from the source Import Set table to the destination Target table. | • Maps source values (e.g., u_first_name) to target fields (e.g., first_name). |

| **Row evaluation prior to commit** | **Coalesce Configuration** | Evaluates if the predefined unique identifier (e.g., Email address) already exists in the target database. | • If match: Updates existing record.

• If no match: Inserts new record. |

| **Row formatting/validation required** | **Transform Script (onBefore / Field-level)** | Executes JavaScript to validate, reject, or reformat specific field values (e.g., standardizing date formats) before insertion. | • Sets formatted value.

• Optionally skips erroneous rows (ignore = true). |

| **Transform run completes** | **Transform History G Logs** | Compiles system processing data, recording exactly what happened to each row from the spreadsheet. | • Displays counts for Inserts, Updates, Ignored, and Errors.

• Provides detailed error messages for troubleshooting. |

**4. Technology Stack G Technical Specifications**

| **Architectural Parameter** | **Technical Specification** |
|---|---|
| **Platform G Target Table** | ServiceNow ITSM Platform \| Target Table (e.g., incident, sys_user) C Staging Table (Import Set) |
| **Application Scope** | Global Scope (or specific Custom Application Scope) |
| **Import Components** | Data Sources, Import Sets, Transform Maps, Field Maps, Choice Actions |
| **Server-side Script Types** | Transform Scripts (onStart, onBefore, onAfter, onComplete, Field-level) |
| **Duplicate Prevention** | Coalesce Field matching logic |
| **Server-side API Methods** | source.getValue(), target.setValue(), ignore = true, error_message, log.info() |
| **Supported File Formats** | Excel (.xlsx, .xls), CSV (Comma-Separated Values) |
| **Execution Context** | Server-side Transform Engine (Asynchronous/Batch database processing) |

# Project Design Phase

## Proposed Solution Template

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 2 Marks |

**Proposed Solution Template**

| S.No. | Parameter | Description |
|---|---|---|
| 1 | **Problem Statement (Problem to be solved)** | Bulk inserting or updating records in ServiceNow from external spreadsheets often suffers from incomplete data, formatting mismatches, and duplicated entries when relying on manual entry or unmapped imports. |

This lack of an automated staging and validation process leads to corrupted databases, orphaned reference data, inaccurate reporting, and massive administrative time wasted on data cleanup. |

| **2** | **Idea / Solution description** | Implementation of an automated, staged data import pipeline using ServiceNow Import Sets and Transform Maps to enforce data integrity at the database level:

• **Data Sources G Import Sets:** Loads raw spreadsheet data into a secure, temporary staging table before it touches the live production database.

• **Transform Maps (Field Mapping):** Automatically or manually maps source spreadsheet columns to the correct target table fields.

• **Coalesce Configuration:** Sets unique identifiers (e.g., Email or Asset Tag) to automatically update existing records, preventing duplicates.

• **Choice Action Rules:** Defines strict behaviors (Create, Ignore, Reject) for rows containing un-matched reference data.

• **Transform Scripts:** Utilizes server-side scripts (e.g., onBefore, Field-level) to programmatically clean, format, or validate data (like standardizing dates or text casing) during the transformation process. |

| **3** | **Novelty / Uniqueness** | Combines out-of-the-box declarative mapping capabilities (Mapping Assist) with programmatic server-side Transform Scripts for advanced data manipulation.

Provides a highly secure "buffer" architecture via the Import Set staging table, ensuring that invalid, malformed, or risky spreadsheet data can be caught, reviewed in the Transform Logs, and discarded *before* it can corrupt the live system database. |

| **4** | **Social Impact / Customer Satisfaction** | Drastically improves administrative operational efficiency and overall data reliability for the enterprise.

By eliminating manual data entry, IT staff and data managers are freed from tedious, fatigue-inducing tasks. Stakeholders experience higher satisfaction because critical system data (such as user profiles, asset inventories, or bulk incident creations) is updated rapidly, accurately, and without business disruption. |

| **5** | **Business Model (Revenue Model)** | Internal Enterprise OPEX Optimization C Cost Avoidance Model:

• Eliminates thousands of hours of manual administrative labor, significantly reducing operational expenses associated with data management.

• Prevents costly, time-consuming database remediation and cleanup projects caused by duplicate or malformed records.

• Maximizes ROI on the ServiceNow platform by ensuring pristine data quality, which is critical for accurate analytics, billing, and SLA tracking. |

| **6** | **Scalability of the Solution** | Highly scalable across the entire ServiceNow platform environment.

The Import Set and Transform Map architecture can be easily adapted to populate any core or custom table (e.g., CMDB CI records, User Administration, Hardware Assets, HR profiles).

It easily scales horizontally to handle massive datasets containing hundreds of thousands of rows, and can be easily upgraded into "Scheduled Data Imports" for recurring, fully automated system integrations. |

## Project Design Phase

## Problem – Solution Fit Template

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 2 Marks |

| S.No. | Canvas Element | Details G Description |
|---|---|---|
| 1 | **Customer Segment(s)** | ServiceNow System Administrators, Data Managers, Implementation Specialists, and Business Process Owners who manage large volumes of system data. |
| 2 | **Jobs-To-Be-Done / Problems** | • Bulk importing records (e.g., users, hardware assets, incidents) from external spreadsheets into ServiceNow. |

• Mapping external data columns to internal database fields accurately.

• Preventing duplicate records and ensuring data formatting matches system requirements. |

| **3** | **Triggers** | A need to onboard hundreds of new employees, load new IT asset inventory, migrate data from a legacy system, or perform bulk updates provided by stakeholders via spreadsheets. |

| **4** | **Emotions: Before / After** | • **Before:** Overwhelmed by the prospect of manual data entry, frustrated by duplicate record creation, and anxious about corrupting the live database with poorly formatted data.

• **After:** Confident, efficient, and in control due to a secure, automated staging and mapping process that guarantees data integrity. |

| **5** | **Available Solutions** | Manual data entry (typing records one-by-one), writing complex and costly custom API integration scripts from scratch, or insecure copy-pasting. |

| **6** | **Customer Constraints** | • Tight deadlines for data migration or bulk updates.

• Source spreadsheets often have inconsistent formatting (e.g., varying date formats) and column headers that do not match the ServiceNow database schema. |

| **7** | **Behaviour** | Stakeholders frequently extract data from external systems into messy Excel or CSV files and hand them to admins, expecting immediate, error-free bulk uploads into the live production environment. |

| **8** | **Channels of Behaviour** | • **8.1 Online:** ServiceNow web browser platform (Data Sources, Load Data UI, Transform Maps, Import Logs).

• **8.2 Offline:** Excel (.xlsx) or CSV spreadsheet files exchanged via email, SharePoint, or local drives. |

| **G** | **Problem Root Cause** | Raw spreadsheets lack inherent relational database constraints. Uploading them directly without a staging phase or mapping logic causes field type mismatches, bypasses system validations, and creates duplicated or orphaned records. |

| **10** | **Your Solution** | Utilizing ServiceNow Import Sets to load spreadsheet data into a temporary staging table, applying Transform Maps to correctly map fields, enforcing Coalesce rules to update existing records (duplicate prevention), and applying Transform Scripts for on-the-fly data formatting. |

**Project Planning Logic G Sprint Estimation**

**Implementation of Data Import using Transform Maps (Spreadsheet)**

*TEAM ID : SWTID-2026-*7082

**Agile Core Definitions G Story Point Scale:**

- **Sprint:** A fixed period/duration (e.g., 6 days) in which a team works to complete a set of tasks.
- **Epic:** A large functional body of work related to data importation, broken down into smaller user stories.
- **Story:** A granular development task focused on a specific import configuration, mapping, or transformation script.
- **Story Point:** A relative estimation score representing effort, complexity, and risk (using Fibonacci-based scale: 1, 2, 3, 5).

**Story Point Effort Mapping:**

| Complexity Level | Story Point | Task Description / Context |
|---|---|---|
| **Very Easy Task** | 1 | Basic setup of Data Source or loading a file into an Import Set. |
| **Normal Task** | 2 | Straightforward Field Mapping (using Mapping Assist) or setting basic Choice Action rules. |
| **Moderate Task** | 3 | Configuring Coalesce rules to update existing records, or writing basic Transform Scripts (e.g., data formatting). |

| Complexity Level | Story Point | Task Description / Context |
|---|---|---|
| **Difficult Task** | 5 | Complex onBefore/onAfter scripts involving cross-table lookups, advanced regex formatting, or multi-field coalescing. |

**Sprint Breakdown G Logic Allocation:**

**Sprint 1**

- **Data Ingestion G Basic Setup (Epic 1)**
    - Setup Data Source C Load file to Import Set (Staging Table) (USN1) -> Story Points: 1
    - Create Transform Map and Auto-Map matching fields (USN2) -> Story Points: 2
- **Data Integrity G Duplicate Prevention (Epic 2)**
    - Configure Coalesce Field (Unique Identifier) (USN3) -> Story Points: 3
    - Define Choice Action behaviors for Reference Fields (USN4) -> Story Points: 2

**Total Story Points in Sprint 1 = 1 + 2 + 3 + 2 = 8**

**Sprint 2**

- **Advanced Data Transformation (Epic 3)**

- Write onBefore Transform Script to standardize incoming date formats (USN5) -> Story Points: 3

- Write Field-level Script to capitalize First/Last names (USN6) -> Story Points: 2

- **Testing G Validation (Epic 4)**

  - Run test import, review Transform logs, and verify Target Table data integrity (USN7) -> Story Points: 3

**Total Story Points in Sprint 2 = 3 + 2 + 3 = 8**

**Velocity Calculation Logic:**

Total Story Points = Sprint 1 (8) + Sprint 2 (8) = 16 Story Points

Number of Sprints = 2

Velocity = Total Story Points Completed / Number of Sprints

Velocity = 16 / 2 = 8 (Story Points per Sprint)

**Your team's velocity is 8 Story Points per Sprint.**

**Ideation Phase**

**Empathize G Discover**

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 4 Marks |

**Empathy Map Canvas:**

Project-context empathy map for users responsible for bulk importing spreadsheet data into ServiceNow. The map focuses on the documented challenges of manual data entry, duplicate record creation, formatting mismatches, and their effect on database integrity and operational efficiency.

| | |
|---|---|
| USER / PERSONA | ServiceNow Administrator / Data Manager |

**What does the user THINK G FEEL?**

- Needs data imports to be fast, secure, and 100% accurate.
- May be concerned that poorly formatted spreadsheet data could corrupt the live database or trigger incorrect notifications.
- Needs the system to intelligently handle duplicates and missing reference data without manual intervention.
- May feel pressure when stakeholders demand large volumes of data (like user lists or hardware assets) be uploaded against tight deadlines.

**What does the user SEE?**

- Raw spreadsheets (Excel/CSV) containing hundreds or thousands of rows with inconsistent formatting.
- ServiceNow setup interfaces including Data Sources, Import Set (staging) tables, Transform Maps, and Mapping Assist.
- Transform History logs detailing the number of row Inserts, Updates, Ignored, and Errors.

**What does the user HEAR?**

- Requests from management or other departments to "bulk upload this spreadsheet into the system."
- Complaints from users about duplicate records or missing information resulting from previous manual data entries.
- Best-practice guidance emphasizing database integrity and the importance of testing imports in a sub-production environment first.

**What does the user SAY G DO?**

- Prepares, reviews, and cleans Excel/CSV files prior to upload.
- Creates Data Sources to ingest files into temporary Import Set tables.
- Configures Transform Maps to link source columns to target database fields.
- Defines "Coalesce" fields to establish unique identifiers for updating existing records.
- Executes the import and monitors the Transform History to troubleshoot errors.

**PAIN**

- Spreadsheet column headers and data types often do not match the target ServiceNow database schema.
- High risk of accidentally creating duplicate records if unique identifiers aren't matched properly.
- Records failing to import because spreadsheet values don't exist in ServiceNow Reference or Choice fields.
- Tedious, time-consuming cleanup required when an unmapped import goes wrong.

**GAIN**

- Visual, automated column matching using the Mapping Assist tool.
- "Coalesce" functionality safely updating existing records instead of creating duplicates.
- Transform Scripts (e.g., onBefore) that automatically clean and format incoming text or dates on the fly.
- Detailed Transform Logs that pinpoint exactly which rows failed and why, making error correction easy.

- Massive time savings and guaranteed data integrity compared to manual data entry.

**Ideation Phase**

**Define the Problem Statements**

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Import Data using Transform Maps (Spreadsheet) |
| Maximum Marks | 2 Marks |

**Customer Problem Statement:** The problem statement is framed from the perspective of the administrators or data managers handling bulk data loads. It focuses on consistent and accurate data mapping, preventing duplicate records, and validating imported spreadsheet data before it reaches the target tables, as described in the project.

| **Problem Statement (PS)** | **I am (Customer)** | **I'm trying to** | **But** | **Because** | **Which makes me feel** |
|---|---|---|---|---|---|
| **PS-1** | a ServiceNow Administrator / Data Manager | import thousands of records from a spreadsheet into the system quickly and accurately | basic copy-pasting or unmapped imports lead to misaligned, incomplete, or corrupt data | the spreadsheet columns, data types, and formatting do not natively match the ServiceNow database schema | overwhelmed by the manual effort and anxious about causing data integrity issues in the live system. |

| Problem Statement (PS) | I am (Customer) | I'm trying to | But | Because | Which makes me feel |
|---|---|---|---|---|---|
| **PS-2** | a ServiceNow Developer / Implementation Specialist | update existing system records in bulk using a new spreadsheet without creating duplicates | the system creates duplicate entries and frequently rejects rows with unmapped reference data | the system doesn't automatically know which unique identifier to match or how to handle unrecognized dropdown choices | frustrated by tedious post-import data cleanup and concerned about the reliability of our system's reporting. |

**Selected Problem Focus:**

Automate and standardize the bulk importation of spreadsheet data by utilizing Import Sets and Transform Maps to accurately map source columns to target fields, enforce "Coalesce" rules to prevent duplicate records, and manage reference data behaviors to ensure high-quality data ingestion.

**Project Design Phase-II**

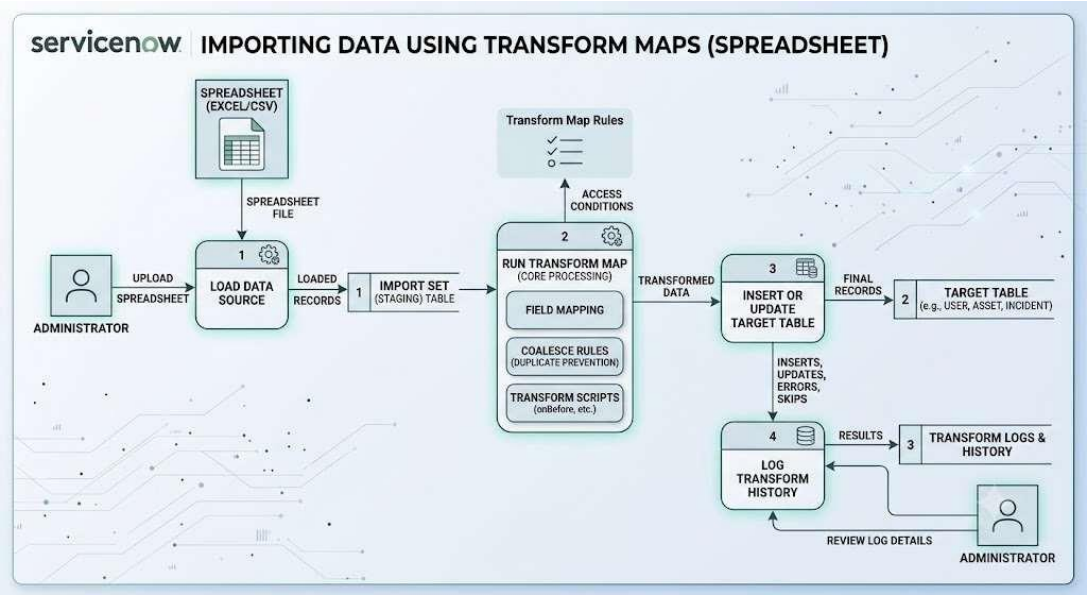
**Data Flow Diagram G User Stories**

| | |
|---|---|
| Date | 30 September 2026 |
| Team ID | SWTID-2026-7082 |
| Project Name | Implement Client Script C UI Policy (Incident) |
| Maximum Marks | 4 Marks |

**Data Flow Diagrams:**

**A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.**

**Example: (Simplified)**




**User Stories**

Use the below template to list all the user stories for the product

| User Type | Functional Requirement (Epic) | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|---|
| **ServiceNow Administrator** | Data Loading | USN-1 | As an admin, I can upload a spreadsheet into a Data Source so that raw data is ingested into the system. | Spreadsheet data successfully loads into the Import Set (staging) table without file format errors. | High | Sprint-1 |
| | Field Mapping | USN-2 | As an admin, I can create a Transform Map to link staging table columns to target table fields. | Auto-mapping correctly links matching names; remaining fields can be mapped manually using the Mapping Assist tool. | High | Sprint-1 |

| User Type | Functional Requirement (Epic) | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|---|
| | Duplicate Prevention | USN-3 | As an admin, I can set a unique identifier (e.g., Email or ID) as a 'Coalesce' field. | If a record with that identifier exists, it updates the record. If not, it creates a new record. No duplicates are created. | High | Sprint-1 |
| | Reference Handling | USN-4 | As an admin, I can set a "Choice Action" for reference fields so the system knows what to do with un-matched data. | Un-matched reference data is either ignored, rejected, or creates a new choice based on configuration. | Medium | Sprint-2 |

| User Type | Functional Requirement (Epic) | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|---|
| **ServiceNow Developer** | Data Formatting | USN-5 | As a developer, I can write a Transform Script (e.g., onBefore) to standardize incoming text formats before insertion. | Data like names or dates are reformatted to match system standards before hitting the target table. | Medium | Sprint-2 |
| **Data Owner / Manager** | Error Monitoring | USN-6 | As a data owner, I can view the Transform History logs after an import runs. | I can see a clear count of Inserts, Updates, Ignored, and Error rows with reasons for any failures. | High | Sprint-1 |

| User Type | Functional Requirement (Epic) | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|---|
| | Execution | USN-7 | As a data owner, I want to execute the Transform process securely without needing full admin access to the database. | I can trigger the data load and transform without impacting live system performance or bypassing security rules. | Low | Sprint-3 |